Bachelorarbeit im Studiengang Computerlinguistik (B.A.)
an der Heinrich-Heine-Universität Düsseldorf

Optimizing RAG Pipelines for Small Language Models

Evaluating Document Preprocessing, Chunking, and Query Expansion on Structured Administrative Data

vorgelegt von:

Daniel Gelderblom
Otto-Hahn-Straße 118
40591 Düsseldorf
Daniel.Gelderblom@hhu.de
Matrikelnummer: 3131057

Abgabedatum:	21. September 2026
Erstgutachterin:	Prof. Dr. Wiebke Petersen
Zweitgutachter:	David Arps

Contents

List of Figures

List of Tables

1 Introduction

Since the publication of OpenAI's GPT-3 (OpenAI 2020), Large Language Models (LLMs) have gained immense importance. A major problem with LLMs, however, is their tendency to hallucinate. Especially in contexts like user queries on specific documents, factual correctness is vital.

Before the rise of transformer-based LLMs, language models had received all the knowledge during training. Guu et al. (2020) first used retrieval from a knowledge base together with generation during training, thus not needing to train the model on all knowledge. Karpukhin et al. (2020) improved dense retrieval (retrieval based on similarity scores of text embeddings), making it better than keyword-based retrieval and thus completing the foundation for Retrieval Augmented Generation (RAG). RAG (Lewis et al. 2020) combines a (most often dense) retriever with a generational model. The retriever takes documents from a knowledge base relevant to a query and feeds them to the text generator to answer the query.

As context windows have increased to one million tokens, the need for RAG systems for cutting edge proprietary LLMs, while still present, has decreased. However, these LLMs cannot serve every use-case. When data is confidential, privacy is crucial, and only a locally running language model can be considered. In these settings, resource constraints often apply. Thus, only Small Language Models (SLMs) are an option here. SLMs have smaller context windows (262,144 tokens for the very current Qwen3.5-2B). Far more importantly, due to their small size, they suffer much more from lost-in-the-middle problems. Lost-in-the middle describes the phenomenon where transformer-based language models fail drastically when the answer to a question is placed in the middle of a long prompt (N. F. Liu et al. 2024). Thus, performance often drops far before reaching the context limit. For SLMs around 8B parameters, this happens as early as 4000 to 8000 input tokens (Leng et al. 2024). This means that only a few short documents can be part of the prompt, increasing the importance of a well-designed RAG pipeline, and especially of the retrieval step. Improving this step, specifically through document preprocessing, chunking, and query expansion, is what this work focuses on.

Preprocessing is the conversion of documents into a form that is easy for the LLM to understand. Chunking is the technique employed to split the documents, which are all larger than 8000 tokens, into smaller pieces. Query expansion means transforming the query into a form that is easier for the retriever to handle. Thus, the first two parts of the pipeline are only done once for all documents in the offline phase, while query expansion is done for every user input in the online phase (Figure 1).

The task at hand is to create a chat-bot to answer administrative questions from students for the Department of Computational Linguistics. The long-term goal is to have this chat-bot run locally on a small GPU. Therefore, only SLMs can be used as generators. Thus, a RAG system with administrative documents as knowledge base is needed. As these documents are in part structured in tables, document preprocessing is a crucial step.

Thus, there are several goals for this thesis. Firstly, a RAG pipeline is built. This pipeline should be modular, so new components can be easily added and existing components can be easily improved and changed. Secondly, the best set-up for preprocessing the structured documents is determined empirically. Thirdly, the benefits of query expansion are investigated. Fourthly, possible hyperparameters are evaluated for the chunking algorithms and the best chunking method is determined for the context at hand.

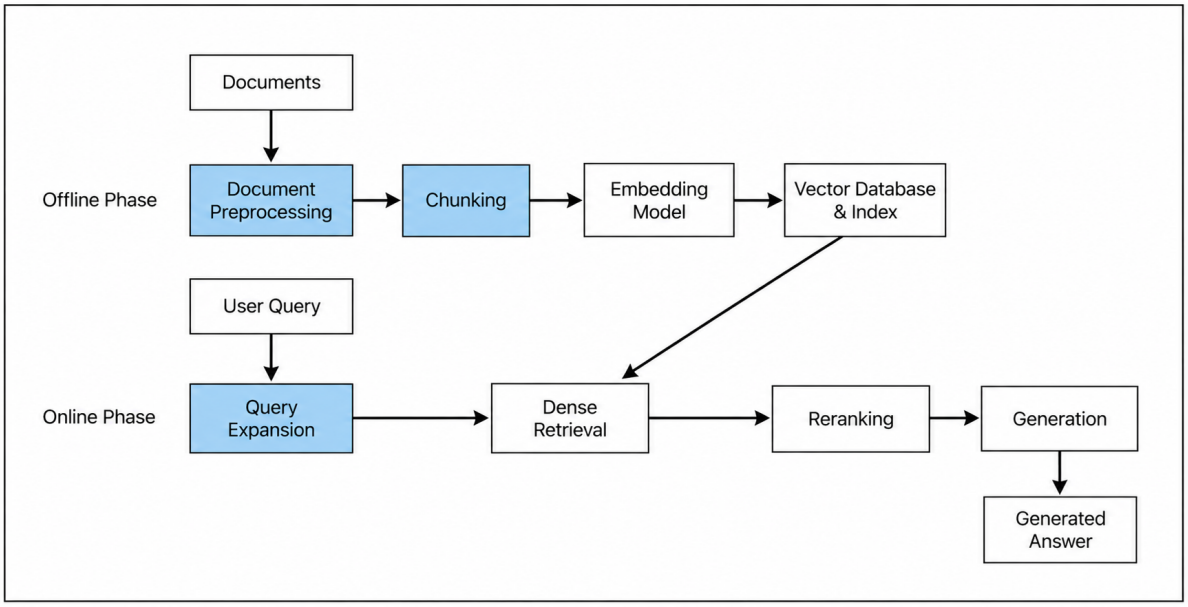

Figure 1: The RAG pipeline

2 Data

2.1 Documents

The RAG-chatbot is based on three administrative documents from the Department of Computational Linguistics at the University of Düsseldorf: Prüfungsordnung (PO), Modulhandbuch (MH) and fachspezifische Prüfungsordnung (fPO)[1]. PO is relatively easy to handle, as it contains all the general information about studying and exams and is structured in paragraphs. MH has to be preprocessed, as all the modules for Computational Linguistics are shown in table-like structures with two columns. For fPO, preprocessing is absolutely crucial as there is a table for every semester containing subjects and their credit points, time required, exams and administrative numbers.

2.2 Evaluation Dataset

To benchmark the performance of the RAG pipeline, a domain-specific evaluation dataset comprising 73 question-answer pairs with human-validated gold answers was constructed. The dataset was created in close collaboration with a student representative from the Department of Computational Linguistics. Candidate questions were generated through a combination of self-written queries and hand-selected suggestions produced by Google Gemini, which were subsequently refined to mirror real-world student inquiries. Table 1 provides representative sample pairs illustrating the different topic areas and difficulty profiles present in the evaluation dataset.

The composition of the dataset reflects strategic advice from the student representative, prioritizing topics that represent frequent points of friction throughout the academic life cycle. Consequently, a strong emphasis was placed on questions typical of the early

[1]The abbreviations are chosen differently in the code due to legacy reasons (Prüfungsordnung was only added later on). In the code, Prüfungsordnung is POges and fachspezifische Prüfungsordnung is PO.

stages of study (Sample pairs 1, 3, 5) and questions about when to take specific courses (Sample pair 1), exams in specific modules (Sample pair 3), and the mandatory internship (Sample pair 4). These topics were combined with broader administrative queries regarding studying in general (Sample pairs 2, 5) and specific questions concerning individual module contents.

To ensure a comprehensive benchmark, the dataset intentionally balances questions across varying levels of retrieval and synthesis difficulty. At the foundational level, it includes easier, direct queries where the answer can be read in one specific location (Sample pairs 2, 4, 5). The dataset also features moderately difficult questions where the target information is located in a single spot but remains hard to find and extract due to dense phrasing or complex tabular structures, particularly information from fPO (Sample pair 3). Finally, the benchmark includes complex queries that require the pipeline to retrieve multiple passages from different sections of a document, interpret them correctly, and synthesize them into a coherent answer, such as assembling a complete first-semester schedule (Sample pair 1).

Table 1: Representative Sample Question-Answer Pairs from the Evaluation Dataset

#	**User Question**	**Gold Answer**
1	Welche Kurse sollte ich im ersten Semester belegen?	Laut dem exemplarischen Studienverlaufsplan solltest du im ersten Semester folgende Kurse belegen: Mathematische Grundlagen, Einführung in die computationelle Logik, Einführung in die Computerlinguistik, Einführung in die Programmierung mit Python, Morphologie und Syntax.
2	Bis wann kann ich mich von einer Klausur ohne Angabe von Gründen abmelden?	Von einer Modulprüfung in Form einer Klausur oder einer mündlichen Prüfung kann sich die Kandidatin oder der Kandidat bis spätestens eine Woche vor dem jeweiligen Prüfungstermin abmelden.
3	Gibt es eine Klausur in Wahrscheinlichkeitstheorie?	Nein, im "Basismodul – Statistische Methoden in der Computerlinguistik", zu dem die Wahrscheinlichkeitstheorie gehört, gibt es gar keine Modulprüfung und somit auch keine Klausur. Es müssen lediglich Studienleistungen erbracht werden.
4	Wann soll ich das Praktikum machen?	Es wird empfohlen, das Berufsfeldpraktikum nach dem vierten oder fünften Semester zu absolvieren.
5	Gibt es eine generelle Anwesenheitspflicht für Vorlesungen?	Nein, bei Vorlesungen kann keine Anwesenheitspflicht festgelegt werden.

In order to measure how well the pipeline copes with questions that can not be answered from the documents, another dataset with 14 unrelated questions was created (see Table 2 for examples). These questions span from questions related to administra-

tive topics of Computational Linguistics (Sample pair 1) over subject-related questions (Sample pair 2) and university-related questions (Sample pair 3) to questions completely out of context (Sample pair 4).

Table 2: Sample Question-Answer Pairs from the Negative Rejection Dataset

#	**User Question**	**Gold Answer**
1	Welche Fristen gelten für die Prüfungsanmeldung der Klausur in Morphologie und Syntax?	Dazu enthalten die bereitgestellten Dokumente keine Informationen.
2	In welchem Jahr wurde die Chomsky-Hierarchie zum ersten Mal publiziert?	Dazu enthalten die bereitgestellten Dokumente keine Informationen.
3	Wie hoch ist der aktuelle Semesterbeitrag an der Heinrich-Heine-Universität Düsseldorf?	Dazu enthalten die bereitgestellten Dokumente keine Informationen.
4	Wie viele Kilometer ist die Sonne durchschnittlich von der Erde entfernt?	Dazu enthalten die bereitgestellten Dokumente keine Informationen.

3 RAG pipeline

3.1 Code Architecture

To facilitate systematic experimentation and ensure future extensibility, the codebase of the RAG pipeline is built upon a strictly modular architecture. A fundamental architectural choice is the strict separation of the pipeline into offline and online execution phases. The offline phase handles the computationally expensive, one-time tasks of document preprocessing, chunking, and vector indexing. The online phase manages real-time interactions, encompassing query expansion, retrieval, reranking, and generation.

Each distinct step of the pipeline is encapsulated within its own module using an object-oriented design approach. The architecture relies on abstract base classes for every pipeline component (e.g., preprocessors, chunkers, retrievers, and generators). Specific algorithms and models are implemented as subclasses that inherit from these base classes. This design pattern ensures that replacing an existing component or integrating a new one—such as introducing a hybrid retriever or a novel chunking algorithm—only requires writing a new implementation of the respective base class, leaving the overarching pipeline logic completely untouched.

To manage the high number of possible pipeline configurations, component selection and hyperparameter choices are handled entirely via a central configuration file. A dedicated Python factory script reads this configuration file and dynamically builds the requested pipeline at runtime by instantiating the specified classes. This allows for rapid component switching and modifications of hyperparameters without the need to modify any source code.

Finally, the architecture is designed for easy maintenance of the knowledge base. New PDF files can be seamlessly integrated into the RAG system by simply placing the raw files into a designated documents directory and registering the new document in

the configuration file. The factory and offline modules will then automatically process, chunk, and index the new data during the next offline run.

3.2 Document preprocessing

As two of the documents contain tables, a direct conversion to raw, unformatted text is not sensible. Min et al. (2024) identified four methods to handle tables for RAG. First, the table can be converted to Markdown. Second, it can be converted to simple sentences using automatic templates. Third, traditional (non-generative) language models can be finetuned for this task. Fourth, LLMs can be used to convert the tables to linearized text. In a comparison of the four strategies, LLM-based conversion performed best, followed by Markdown, which still gave good generation results. Thus, for the RAG pipeline, both Markdown and LLM-based conversion are used. For conversion to Markdown, Docling (Deep Search Team 2024) and conversion by an LLM are tested. Docling is a framework based on pre-trained lightweight vision models and a deep learning model specialized on table conversion. The prompt for the conversion via LLM can be found in Listing 1.

Listing 1: Prompt for conversion to Markdown

```
Convert this document to markdown. Pay attention to table
   structure and the separation of tables and modules.
```

For conversion to text by an LLM, the technique from Min et al. and a self-written prompt are compared. In the paper, the table is first converted into a nested bracket structure and then converted to text by the LLM. The conversion prompt from the paper is translated into German. The self-written prompt (see Listing 2) takes the documents in Markdown and converts them directly to text.[2]

Listing 2: Prompt for Linearization from Markdown to Text

```
Wandle das folgende Markdown-Dokument in klaren,
   informationsdichten Fließtext um, der für eine semantische
    Suchmaschine (RAG-Pipeline) optimiert ist.

Befolge dabei strikt diese Regeln:
1. Informationserhalt: Übernimm ausnahmslos jede Information,
    jede Zahl und jedes Detail.
2. Hohe Entitätsdichte (WICHTIG): Vermeide Pronomen (er, sie,
    es, diese) über Absatzgrenzen hinweg. Wiederhole
   stattdessen immer wieder die konkreten Eigennamen,
   Produktnamen oder Fachbegriffe, damit jeder Absatz auch
   isoliert verständlich bleibt.
3. Tabellen & Listen: Wandle Tabellen und Aufzählungen in
   zusammenhängende Textabsätze um. Formuliere die
   Beziehungen zwischen Spalten/Zeilen in ganzen Sätzen aus.
4. Überschriften als Kontext: Nutze Überschriften, um den
   darauffolgenden Absatz mit einem klaren Kontext-Satz zu
   beginnen (z. B. statt nur "Spezifikationen:" -> "Die
   technischen Spezifikationen des Geräts X umfassen...").
```

[2]Parts of this section have been taken from the previous literature review

```
5. Formatierung: Entferne alle Markdown-Zeichen (*, #, -, |).
    Behalte klare Absatzumbrüche bei, um logische Trennungen
   beizubehalten.

Input Text:
---
{text}
---
```

3.3 Chunking

With early transformer models and the first RAG systems, chunking was only done in a fixed size of tokens, words or characters (Dai et al. 2019; Lewis et al. 2020). For this thesis, Fixed Character Chunking was implemented as a baseline chunker. With Dynamic Token Chunking and Fixed Paragraph Chunking, two other simple deterministic Chunkers were implemented. Dynamic Token Chunking sets a maximum number of tokens and cuts off at the last sentence ending before the limit. Fixed Character Chunking packs a fixed number of paragraphs into one chunk. Fixed Character Chunking and Fixed Paragraph Chunking are used in combination with both preprocessing techniques, while Dynamic Token Chunking is only used with linearized text, as it needs sentences for splitting and does not work with Markdown tables.

A big problem of chunking in general, and especially fixed-size chunking, is the splitting of contexts (context problem). A word or a sentence in one chunk can reference parts of the previous chunk or even a more distant one. This information gets lost in the chunking process. The first solution to this problem, which was derived from the training of BERT (Devlin et al. 2019), was overlapping chunks. Overlap is a certain number of tokens or characters that two adjacent chunks share, giving the beginning and the end of a chunk more context. For Fixed Character Chunking, Fixed Paragraph Chunking, and Dynamic Token Chunking, an overlap is tested in the experiments.

Another solution to the context problem is semantic chunking. This method embeds every sentence and separates two adjacent sentences into different chunks if their similarity is below a certain threshold (Kamradt 2024). However, later studies found that semantic chunking had only a slight, non-significant edge over fixed-size chunking for large contexts, which does not justify the computational overhead (Qu, Tu, and Bao 2025; Bennani and Moslonka 2026). This led to the development of Max-Min-Chunking, an improved semantic chunker achieving significantly better retrieval and RAG accuracy than both previous semantic chunkers and simple fixed-size chunkers. The algorithm uses "dynamic clustering". With Max-Min semantic chunking, a sentence is allocated to the same chunk as the previous one if it shares a higher similarity with any sentence in the current chunk than the pair with the lowest similarity in the chunk. This dynamic threshold can even be dampened or increased by a factor c. It is also getting higher with increasing chunk size through a sigmoid-based function. There is, however, a hard baseline for cosine similarity (fixed_threshold). The Max-Min chunker is tested for linearized text in the chunking experiments.

An alternative approach to deal with the context problem is improving the workflow of chunking. Contextual Retrieval addresses it by prepending contextual text to every chunk, situating it inside the bigger document. This significantly reduces the retrieval failure rate (Anthropic 2024). JinaAI proposed a more profound change to the retrieval

pipeline. Late Chunking embeds all tokens first so the word embeddings have the context of the whole text. Only then are the documents chunked using any chunking algorithm. In the end, mean pooling is done on the vectors in each chunk (Günther et al. 2024). However, both methods are more sensible for in-corpus retrieval, meaning retrieval from a big, thematically heterogenous corpus. For this thesis, there are only three, thematically homogenous documents included, making the task more similar to in-document retrieval (retrieval from one long, homogenous document). Here, contextualized chunking with standard methods is not advised (Zhou et al. 2026). Instead, two structure-based methods enriching table chunks with context were implemented specifically for the documents. The first one, Whole Table Paragraph Chunker, splits the structured Markdown documents into entire tables (for fPO) or modules (for MH). Each table chunk receives the abbreviation legend and its heading as contextualization. The normal, unstructured text in the Markdown documents is split into paragraphs.
The second one, the Split Table Paragraph Chunker, splits semester tables at module boundaries. It also divides module descriptions by individual field lines. This makes retrieval more precise and reduces the context for the LLM. Because context gets lost here, the relevant semester or module headings are prepended to each chunk. Additionally, abbreviations are replaced by their full words, and sub-course letter references are resolved to their actual names. Normal text, like in PO, is split into sentences to stay true to the small chunk size.
With transformer-based LLMs getting more reliable, the models were used for more mechanical tasks like chunking. LumberChunker successfully implemented this approach (Duarte et al. 2024). The document is split into paragraphs. Iteratively, all paragraphs up to a certain token length are fed to an LLM to decide where the content changes, which is where the current chunk is cut. The resulting chunks align closely with manually created chunks and thus perform quite well. For in-document retrieval, LumberChunker performed particularly well in a comparative study (Zhou et al. 2026). Thus, LumberChunker was selected as one of the chunkers for the chunking experiments for linearized text. The prompt from the original LumberChunker paper was translated into German (Listing 3).[3]

Listing 3: LumberChunker translated Prompt

```
Du wirst als Eingabe ein deutsches Dokument mit
   Paragraphen erhalten, die durch 'ID XXXX: <text>'
   identifiziert werden.

Aufgabe: Finde den ersten Paragraphen (nicht der erste),
   bei dem sich der Inhalt im Vergleich zu den vorherigen
    Paragraphen deutlich ändert.

Ausgabe: Gib die ID des Paragraphen mit dem
   Inhaltswechsel im Format zurück: 'Answer: ID XXXX'.

WICHTIGE REGELN:
1. Erkläre NICHT deine Gedankengänge.
2. Gib NUR die ID des Paragraphen mit dem Inhaltswechsel
   im Format zurück: 'Answer: ID XXXX'.
```

[3]Parts of this section have been taken from the previous literature review

```
3. Vermeide sehr lange Gruppen von Paragraphen. Strebe
   ein gutes Gleichgewicht zwischen der Identifizierung
   von Inhaltswechseln und der Beibehaltung handhabbarer
   Gruppen an.

Dokument:
{window_text}
```

3.4 Indexing and Retrieval

After chunking, the chunks are embedded using a sentence transformer model. The model used is jinaai/jina-embeddings-v5-nano, a cutting-edge embedding model with only 239M parameters (Akram et al. 2026). A particularly small model was chosen in order to keep resource usage limited, also for indexing and retrieval. FAISS was used for vector storage (Douze et al. 2024).

3.5 Query expansion

Retrievers suffer from an elementary flaw: the query is structurally very different to the documents. For many models, this is solved during training, so a query is embedded differently than a document (Akram et al. 2026). This still does not completely solve the discrepancy. Query expansion techniques aim to bridge the structural gap by rewriting the query and retrieving against the expanded queries. Two of those techniques are tested and compared to the baseline with no query expansion. The SLM used for generating the RAG answer is also used for query expansion. The prompts for the two techniques are listed in appendix .1.
HyDE (Hypothetical Document Embeddings) is one of the first and most used query expansion techniques (Gao et al. 2023). It uses an LLM to generate a hypothetical answer to the query without using any documents. Afterwards, it retrieves against this generated answer.
Jagerman et al. (2023) compared different query expansion techniques as zero-shot prompts (without examples in the prompt) and few-shot prompts (with some examples in the prompt). The main metric used in the original paper was recall@1K, which is the proportion of all actual relevant items that the system successfully returns in its top 1,000 retrieved results. The prompt with the highest recall was Chain-of-Thought, a prompt that explicitly asks the LLM to give the rationale before answering. This is the second prompt tested for query expansion. Query2doc, another popular method wasn't used because it relies on finetuning the embedding model together with query expansion (Wang, Yang, and Wei 2023).

3.6 Reranking

Reranking improves retrieval by retrieving k documents, reranking the retrieved results and only returning the top-n documents (Nogueira and Cho 2019), thus $k > n$. The reranker is more computationally expensive than the retriever, as it uses cross-encoding instead of a simple vector similarity computation. Cross-encoding means processing the

query and a retrieved document jointly through a transformer model. Afterwards, a classification token is returned to indicate the similarity between the query and the document. The reranker used here is jinaai/jina-reranker-v3.

3.7 Generation

Due to practical requirements for later deployment, a very small language model was chosen for generation. The model is Qwen/Qwen3.5-2B. This model was chosen due to its popularity and its good performance on multilingual benchmarks compared to its size (Qwen 2026). Some early tests were done with Qwen/Qwen3.5-0.8B, but the output in German was too unreliable.

3.8 Left-out Methods

Several RAG frameworks created by previous papers were tested or considered but in the end not included for various reasons.
TableRAG (S.-A. Chen et al. 2024) uses an LLM to create queries for pandas. However, this does not take mixed documents into consideration, where there are both text and tables. Another TableRAG framework based on SQL (Yu, Jian, and Chen 2025) takes both into account. However, the integration for the experiments and especially for later deployment was too complex due to database connections and a complex architecture.
TabRAG (Si et al. 2025) separates and includes both text and tables into RAG using a vision learning model. This was tested on the faculty GPU server, but the model turned out to be too large for the resources available on the server.
MiniRAG (Fan et al. 2025) is a graph-based framework specifically designed for SLMs. It extracts entities and relationships from the documents and links entities to context. With the university documents, this method was hardly functionable. Some entities could be extracted, but the algorithm detected no relationships between them. Thus, the elaborate retrieval algorithm could did work and the generation results were worse than even for simple chunkers.

4 Experiments

4.1 Evaluation Setup

Due to the large variety in preprocessing and chunking techniques, the retrieved chunks of different experiments are very different in structure and content. Thus, classic retrieval evaluation techniques like determining precision and recall from token spans fail here or would come with a huge manual overhead. Instead, an LLM is used to evaluate retrieval and generation results (LLM-as-a-judge). Previous studies using GPT-4 have shown that LLMs achieve an agreement rate on par with the inter-annotator agreement for this task (Y. Liu et al. 2023; Zheng et al. 2023). In order to counter the problem of LLMs favoring their own answers (W.-L. Chen et al. 2025), GPT 4o-mini is used for evaluation, as most of the preprocessed chunks were generated by Google Gemini.
The metrics in use stem from RAGAS (Es et al. 2024). The original RAGAS paper introduced LLM-as-a-judge for RAG specifically with three metrics. Faithfulness judges, if the answer can be inferred from the context. This is done by splitting the answer into statements and dividing the number of grounded statements by the total number

of statements. Answer relevancy judges, if the answer addresses the query. For this, questions are generated based on the LLM response and then embedded. The score is the average cosine similarity of the questions and the original query. Context Relevance judges if the context only contains relevant information, but is dependent on a sentence-based structure of the chunks and thus not used here.
Instead, two other metrics from the RAGAS package are used. The first is context recall. This metric measures how many of the relevant chunks were successfully retrieved. It divides the number of claims in the reference supported by retrieved chunks by the total number of claims in the reference. The second is answer correctness. This measures the correctness of the final generated answer by combining a factual F1 score with embedding similarity. The F1 score is calculated as follows:

$$\text{F1 Score} = \frac{|\text{TP}|}{(|\text{TP}| + 0.5 \times (|\text{FP}| + |\text{FN}|))}$$

TP (true positives) are facts extracted from the generated answer that are present in the reference answer. FP (false positives) are facts present in the generated answer that are not present in the reference answer. FN (false negatives) are facts present in the reference answer that are not present in the generated answer. Answer correctness is the main metric for evaluating the pipeline. The goal of this paper is to find a pipeline configuration that works well in combination with the SLM and answer correctness tests the overall result. However, the other metrics can help to identify problems with certain pipeline configurations.
An important attribute of RAG pipelines which cannot be captured by the previous four metrics is negative rejection. This is the ability to reject questions which cannot be answered with the given context. For the RAGAS framework, a metric with a custom prompt was implemented to measure negative rejection (see Listing 4). This metric was used for the second dataset containing unanswerable queries.

Listing 4: Negative Rejection Prompt

```
Did the model reject the query as not answerable from the
    given context?
```

4.2 Pipeline Configuration

The RAG pipeline encompasses numerous configurable components and hyperparameters. Because evaluation using LLM-as-a-judge is computationally and resource-intensive, exhaustive grid searches across the entire parameter space are impractical. To address this, four preliminary sequential experiments are conducted to independently optimize document preprocessing, chunking hyperparameters, context size, and query expansion prior to the main evaluation. In each stage, the optimal parameters identified in preceding steps are adopted. For instance, following the preprocessing benchmark, only the top-performing preprocessors are evaluated in subsequent pipeline stages. To conserve computational resources during parameter tuning, these preliminary experiments are executed on a representative, thematically balanced subset of 15 question-answer pairs drawn from the primary evaluation dataset.

4.3 Document Preprocessing

In the first experiment, document preprocessing strategies are evaluated in two stages. First, Markdown conversion produced by Gemini is compared against output from Docling (Deep Search Team 2024). Second, the superior Markdown preprocessor is combined with the custom prompt for text linearization and evaluated against the multi-step table linearization method from Min et al. (2024). The top-performing approach from each comparison is selected for all subsequent pipeline configurations. To prevent biasing the evaluation toward table-specific chunking strategies, the Fixed Paragraph Chunker (size 1, overlap 0) is employed as a baseline due to its general applicability to document structures. Because individual paragraphs can be relatively long (up to 600 tokens), the retrieval parameters are fixed at $k = 9$ retrieved documents (top_k) and $n = 3$ reranked documents (top_n).

4.4 Grid Search for Chunking Hyperparameters

For the deterministic strategies (Fixed Character, Fixed Paragraph, and Dynamic Token Chunking), chunk size and overlap parameters are systematically tuned such that individual chunks remain below 1500 tokens in most cases. Fixed Character and Fixed Paragraph strategies are tuned independently for Markdown and linearized documents to accommodate structural differences. For Fixed Paragraph Chunking, an overlap of 1 paragraph is evaluated for chunk size 2. For Fixed Character and Dynamic Token Chunking, a 10% overlap is compared against a zero-overlap baseline.

For Max-Min Chunking, the hard cosine threshold ($hard\_threshold$) and dampening parameter (c) are optimized via grid search. Initial exploratory tests revealed that the baseline hyperparameter settings proposed by Qu, Tu, and Bao (2025) caused infinite aggregation loops resulting in single, oversized chunks. When similarities near 0.6 occurred early in a chunk while the sigmoid-based dynamic threshold remained low, multiple moderately dissimilar sentences were aggregated together. This artificially expanded the semantic diversity of the chunk, which in turn increased the maximum similarity score for subsequent candidate sentences and further lowered the dynamic threshold. This behavior is largely driven by the high thematic homogeneity of the university administrative documents. To prevent this runaway effect, candidate values for c and $hard\_threshold$ were restricted to strictly higher ranges than those reported in the original paper.

4.5 Context Size and Reranking Thresholds

Determining the optimal number of retrieved (top_k) and reranked (top_n) chunks presents a structural trade-off. While maintaining fixed values for top_k and top_n across all configurations is straightforward, it leads to uneven context lengths during generation due to large variations in average chunk size across different chunking strategies. To establish fair comparisons and respect context limitations, a target token limit of 1500 tokens is enforced, dynamically deriving top_n and top_k from the average chunk size (S_{avg}):

$$N_{\text{top}} = \max\left(1, \left\lfloor \frac{1500}{S_{\text{avg}}} \right\rfloor\right), \quad K_{\text{top}} = 3N_{\text{top}} \tag{1}$$

The 1500-token threshold is selected because context performance in open-weight models around 8B parameters degrades significantly between 4000 and 8000 input tokens (Leng

et al. 2024). Setting a conservative budget for the 2B generator model minimizes lost-in-the-middle phenomena.

However, a strict token cap may disadvantage fine-grained chunking strategies designed to provide short, highly targeted context. To evaluate whether variable context lengths improve performance, a similarity thresholding mechanism is tested. Under this setup, fewer chunks are passed to the generator if the cross-encoder similarity scores of lower-ranked chunks fall below a specific cutoff. This mechanism is evaluated across multiple score thresholds on both paragraph-level Markdown text and fine-grained Split Table chunks.

4.6 Query Expansion

The two query expansion techniques—Hypothetical Document Embeddings (HyDE) and Chain-of-Thought (CoT) prompting—are evaluated against a baseline without query expansion. To maintain low resource requirements, the generator SLM (Qwen3.5-2B) is also used to generate the expanded queries. Following the setup of the preprocessing experiment, Paragraph Chunking is used as the baseline chunking method for this evaluation.

4.7 Main Experiment: Preprocessor and Chunker Comparison

In the final experiment, all viable combinations of preprocessors and chunkers—configured with the optimal hyperparameters established in the preceding steps—are evaluated on the complete dataset of 73 question-answer pairs as well as the 15-item negative rejection dataset. Combinations that are structurally incompatible (such as applying table-aware chunkers to linearized text) are excluded from the main comparative benchmark. Two experiments are run as baselines for comparison. The first one uses no preprocessing and no chunking. Thus, the SLM is given all three documents as raw text without any formatting. For the second baseline experiment, the documents are preprocessed to linearized text by the optimized preprocessor setup (GeminiMarkdownProcessor+DirectLLMProcessor), but are not chunked. These baseline experiments serve to make sure that the introduction of chunking and RAG has a measurable effect on generation results.

5 Results

5.1 Preprocessing choices

GeminiMarkdownProcessor outperforms DoclingMarkdownProcessor for Markdown conversion. Together with DirectLLMProcessor, it also outperforms PaperLLMProcessor for text linearization (see table 4).

Table 3: Comparison of Document Preprocessors

Preprocessor	**Answer Accuracy**
DoclingMarkdownProcessor	0.454
GeminiMarkdownProcessor	0.485
GeminiMarkdownProcessor+DirectLLMProcessor	0.531
PaperLLMProcessor	0.439

5.2 Chunking Parameters

The results for the hyperparameter grid search across chunking strategies and preprocessors are presented in Table 4. For Fixed Character Chunking on Markdown documents (GM), larger chunk sizes without overlap yielded the highest performance, peaking for a chunk size of $S = 1500$ characters. Introducing an overlap consistently degraded performance across all tested chunk sizes for Markdown, most noticeably at $S = 500$, where adding a 50-character overlap dropped accuracy from 0.6612 to 0.4191. For Fixed Paragraph Chunking on Markdown, a chunk size of $S = 2$ paragraphs without overlap achieved the highest score, outperforming both smaller ($S = 1$) and larger ($S = 3$) paragraph windows as well as configurations with paragraph overlap ($S = 2, O = 1$).

When applied to linearized text (GM+LLM), Fixed Paragraph Chunking performed best at a smaller size of $S = 1$ paragraph, with performance declining as the paragraph count or overlap increased. For Fixed Character Chunking on linearized text, a smaller chunk size with overlap ($S = 500, O = 50$) achieved the best result. For Dynamic Token Chunking, a smaller context window with a 10% overlap ($S = 150, O = 15$) proved optimal. Finally, for Max-Min Chunking, tuning the hard threshold (τ) and dampening parameter (c), $\tau = 0.75$ and $c = 1.3$ achieved the best results for this strategy. Both values stand in the middle of the ranges for their respective parameter and thus confirm the testing range.

Table 4: Grid Search Results for Chunking Hyperparameters

Prep.	Chunker	Params	Acc.	Prep.	Chunker	Params	Acc.
GM	FixedChar	S=500, O=0	0.6612	GM+LLM	MaxMin	τ=0.70, c=1.3	0.4982
GM	FixedChar	S=500, O=50	0.4191	GM+LLM	MaxMin	τ=0.70, c=1.4	0.5204
GM	FixedChar	S=1000, O=0	0.5598	GM+LLM	MaxMin	τ=0.70, c=1.5	0.5247
GM	FixedChar	S=1000, O=100	0.5967	GM+LLM	MaxMin	τ=0.75, c=1.2	0.5720
GM	FixedChar	S=1500, O=0	**0.6811**	GM+LLM	MaxMin	τ=0.75, c=1.3	**0.5861**
GM	FixedChar	S=1500, O=150	0.5887	GM+LLM	MaxMin	τ=0.75, c=1.4	0.5440
GM	FixedPara	S=1, O=0	0.6166	GM+LLM	MaxMin	τ=0.75, c=1.5	0.5819
GM	FixedPara	S=2, O=0	**0.6274**	GM+LLM	MaxMin	τ=0.80, c=1.2	0.5392
GM	FixedPara	S=2, O=1	0.5939	GM+LLM	MaxMin	τ=0.80, c=1.3	0.5524
GM	FixedPara	S=3, O=0	0.5702	GM+LLM	MaxMin	τ=0.80, c=1.4	0.5496
GM+LLM	FixedPara	S=1, O=0	**0.6605**	GM+LLM	DynToken	S=150, O=0	0.6058
GM+LLM	FixedPara	S=2, O=0	0.5988	GM+LLM	DynToken	S=150, O=15	**0.6386**
GM+LLM	FixedPara	S=2, O=1	0.5421	GM+LLM	DynToken	S=250, O=0	0.6084
GM+LLM	FixedPara	S=3, O=0	0.5275	GM+LLM	DynToken	S=250, O=25	0.5738
GM+LLM	FixedChar	S=500, O=0	0.6019	GM+LLM	DynToken	S=500, O=0	0.5975
GM+LLM	FixedChar	S=500, O=50	**0.6184**	GM+LLM	DynToken	S=500, O=50	0.4894
GM+LLM	FixedChar	S=1000, O=0	0.5313				
GM+LLM	FixedChar	S=1000, O=100	0.5716				
GM+LLM	FixedChar	S=1500, O=0	0.5619				
GM+LLM	FixedChar	S=1500, O=150	0.5261				

Note: GM = GeminiMarkdownProcessor, LLM = DirectLLMProcessor, S = Size, O = Overlap, τ = Threshold.

5.3 Context Size and Reranking Threshold

The performance evaluation across reranking thresholds is presented in Table 5. Overall, applying a similarity threshold yielded mixed effects depending on the chunking strategy and preprocessing format. For several configurations, introducing a moderate threshold of 0.05 improved answer correctness compared to the baseline without thresholding (threshold = 0.00). Specifically, Whole Table Chunking peaked at 0.647, Dynamic Token

Chunking reached 0.639, Max-Min Chunking improved to 0.584, and the LLM Chunker increased to 0.526 under the 0.05 cutoff.

Conversely, for standard Paragraph and Character chunkers across both Markdown and Direct preprocessing, introducing a threshold generally led to a decline in answer correctness compared to the 0.00 baseline. An exception to this pattern was observed with the Split Table Chunker, which performed poorest at the 0.05 threshold (0.549) but achieved its highest accuracy at a stricter threshold of 0.10 (0.591). Across most other strategies, however, raising the threshold to 0.10 resulted in noticeable performance drops, most prominently seen in Whole Table Chunking (falling to 0.544) and Dynamic Token Chunking (falling to 0.518).

Table 5: Comparison of Reranking Thresholds for each Preprocessor/Chunker Combination

			Reranking Threshold		
Preprocessing	**Chunker**	**Avg. Tokens**	**0.00**	**0.05**	**0.10**
Markdown	Paragraph	221.9	**0.627**	0.593	0.601
	Character	385.2	**0.649**	0.584	0.580
	Whole Table	97.4	0.645	**0.647**	0.544
	Split Table	40.4	0.579	0.549	**0.591**
Linearized	Paragraph	375.6	**0.598**	0.592	0.524
	Character	121.6	**0.600**	0.568	0.579
	Dynamic	217.5	0.630	**0.639**	0.518
	LLM Chunker	752.2	0.504	**0.526**	0.516
	Max-Min	77.2	0.576	**0.584**	0.578

5.4 Query Expansion

There was no clear best strategy for query expansion across all preprocessing and chunking strategies (Table 6). However, some trends could be observed. Overall, Chain-of-Thought (CoT) prompting proved to be a more effective query expansion technique than Hypothetical Document Embeddings (HyDE), achieving the highest answer correctness in four out of the nine evaluated preprocessor-chunker pairings compared to only one for HyDE. CoT achieved notable improvements on Direct LLM preprocessed documents when paired with Character Chunking (0.677 vs. 0.612 baseline) and Paragraph Chunking (0.646 vs. 0.639 baseline), as well as on Markdown documents with Whole Table Chunking (0.647 vs. 0.620 baseline).

Furthermore, query expansion exhibited a higher overall impact when applied to Markdown preprocessed documents than to linearized text. For Markdown documents, query transformation led to substantial variations in performance: HyDE yielded a performance gain for Paragraph Chunking (0.589 vs. 0.556 baseline), whereas CoT caused a marked drop (falling from 0.556 to 0.501). Conversely, for linearized text, applying query expansion had a much more subtle effect. Even in cases where the baseline without processing achieved the top score—such as with Dynamic Token Chunking (0.636 baseline vs. 0.629 CoT and 0.624 HyDE), LLM Chunking (0.507 baseline vs. 0.502 HyDE and

0.489 CoT), and Max-Min Chunking, the margin between the baseline and the expansion techniques remained very narrow.

Table 6: Evaluation of RAG Pipeline Answer Correctness Across Query Processors.

		Query Processor		
Preprocessing	**Chunker**	**No Processing**	**HyDE**	**CoT**
Markdown	Paragraph	0.556	**0.589**	0.501
	Character	**0.689**	0.660	0.603
	Whole Table	0.620	0.640	**0.647**
	Split Table	**0.551**	0.514	0.459
Linearized	Paragraph	0.639	0.588	**0.646**
	Character	0.612	0.579	**0.677**
	Dynamic	**0.636**	0.624	0.629
	LLM Chunker	**0.507**	0.502	0.489
	Max-Min	0.585	0.585	**0.595**

5.5 Main Experiment

The main experiment demonstrates the clear advantage of employing proper preprocessing and chunking strategies. As shown in Table 7, the baseline without any preprocessing or chunking achieved an answer correctness of only 0.423. Introducing preprocessing to create linearized text without chunking improved the results solidly to an answer correctness of 0.485. However, combining preprocessing with chunking yielded the best outcomes, with all tested combinations achieving an answer correctness between 0.51 and 0.57, outperforming both baselines.

Within the Markdown preprocessing configurations, the Whole Table and Paragraph chunkers demonstrated very similar and strong results across all metrics. Both achieved good answer correctness and high context recall, with the Whole Table strategy reaching the highest context recall (0.902) among all nine evaluated strategies. Conversely, the Split Table and Character chunking methods performed similarly poorly on Markdown documents, yielding both low answer correctness and low context recall.

For linearized text preprocessing, Character and Max-Min Chunking emerged as the best-performing chunkers overall. Character chunking on linearized text achieved the highest answer correctness (0.577) and the highest faithfulness (0.740), alongside a slightly above-average context recall. Max-Min Chunking also performed very well, pairing high faithfulness with the highest answer relevancy across all experiments (0.711), though its context recall remained mid-range. In contrast, LLM chunking showed generally poor performance on linearized text, resulting in low answer correctness (0.517) and very low context recall. Furthermore, while the Paragraph chunker on linearized text achieved high context recall and faithfulness, this did not translate to strong final generation quality, resulting in an answer correctness of only 0.533.

Comparing the two preprocessing paradigms, faithfulness was generally higher for linearized text than for Markdown. With the exception of the underperforming LLM chunker, all linearized text strategies achieved higher faithfulness scores than all tested Markdown strategies.

Another notable observation is the behavior of the Dynamic Token chunker on linearized text. It achieved high context recall but relatively low faithfulness, leading to a mid-range answer correctness (0.544). However, it managed by far the highest negative rejection rate (0.929).

More broadly, the negative rejection rate appeared to be quite independent of answer correctness. For instance, Character chunking on Markdown had very low answer correctness but a very high negative rejection rate (0.857). Conversely, Character chunking on linearized text achieved the highest answer correctness but the lowest negative rejection rate (0.643). Among all configurations, Max-Min Chunking stood out as the only strategy that effectively balanced both high answer correctness and a strong negative rejection rate.

Table 7: Evaluation of RAG Pipeline Across Preprocessing and Chunking Strategies Across All Metrics.

Preprocessing	**Chunker**	**CR**	**Faith.**	**AR**	**AC**	**NRR**
No Preprocessing	No Chunking				0.423	0.571
Markdown	Paragraph	0.856	0.683	0.658	0.553	0.786
	Character	0.775	0.672	0.689	0.515	0.857
	Whole Table	**0.902**	0.663	0.636	0.556	0.786
	Split Table	0.749	0.687	0.688	0.515	0.714
Linearized	No chunking				0.485	0.643
	Paragraph	0.862	0.737	0.656	0.533	0.786
	Character	0.839	**0.740**	0.666	**0.577**	0.643
	Dynamic	0.861	0.689	0.627	0.544	**0.929**
	LLM Chunker	0.745	0.671	0.598	0.517	0.857
	Max-Min	0.829	0.737	**0.711**	0.570	0.857

Note: CR = Context Recall, Faith. = Faithfulness, AR = Answer Relevancy, AC = Answer Correctness, NRR = Neg. Rejection Rate.

6 Discussion

6.1 Preprocessing

The first part of the preprocessing experiment hints to the fact that LLMs can outperform elaborate frameworks at complicated tasks. Docling was specifically designed for converting documents containing tables to markdown. Still, Gemini performed better with a simple one line prompt.

The good results of the text linearization technique from Min et al. (2024) could not be reproduced. Instead, also here, a custom prompt using the Markdown documents as input achieved a better answer correctness. One explanation for this could be the direct translation of the table transformation prompt from the paper into German, which might not have been understood properly by the model. Another possible explanation is that the prompt was highly fitted to the datasets used in the paper. As the university documents look significantly different, the combination of pdf-to-text and prompt from the paper might just not work here.

6.2 Chunking Parameters

The grid search results reveal a clear divergence in optimal chunking strategies depending on the preprocessing format, highlighting the different ways Small Language Models (SLMs) process structured versus dense text.

For Markdown documents, applying zero overlap proved to be the most effective approach. Introducing overlap in Markdown duplicates unnecessary structural syntax (such as table delimiters and formatting tokens) across chunks. This not only wastes the limited context window but also feeds the SLM redundant structural noise that is difficult to process. Furthermore, Fixed Paragraph Chunking on Markdown performed best with a chunk size of two paragraphs. With a size of one, a table's explanatory header or introductory paragraph is always separated from the table itself. A chunk size of two often prepends some necessary context to the table. For Fixed Character Chunking, a large chunk size was important to limit table fragmentation.

In contrast, when using linearized text, Fixed Paragraph Chunking peaked with a chunk size of just one paragraph. The custom prompt utilized by the `DirectLLMProcessor` explicitly enforces "high entity density" and explicit contextual relationships. Consequently, a single paragraph is already heavily saturated with information. Feeding multiple data-dense paragraphs to the 2B-parameter generator overloads its context processing capabilities, triggering the "lost-in-the-middle" phenomenon where the model fails to isolate the correct facts.

For deterministic, sub-paragraph chunkers operating on linearized text—namely Fixed Character and Dynamic Token chunking—smaller chunk sizes consistently yielded better results. Smaller sizes generate more specialized, targeted chunks, allowing the retriever to supply the SLM with highly specific facts. However, because these chunkers are fundamentally blind to natural semantic boundaries (lacking paragraph or Markdown notation cues), their cuts are highly arbitrary. Therefore, an overlap is strictly necessary to act as a semantic bridge. This is particularly crucial for Character chunking, where the inherent risk of mid-word or mid-sentence text fragmentation makes an overlap essential for maintaining contextual coherence.

Finally, the optimization of the Max-Min Chunker confirmed the initial hypothesis regarding the algorithm's threshold parameters. Due to the high thematic homogeneity of the administrative documents, utilizing the low dampening factor (c) and hard threshold from the original paper led to degenerate chunking behavior, where the algorithm grouped vast sections of text into unusable, oversized chunks. Conversely, setting these parameters too high triggered the opposite degeneration: almost every sentence was isolated into its own chunk, entirely defeating the purpose of semantic aggregation. The grid search successfully identified the algorithmic sweet spot, with RAG performance declining predictably when the thresholds were set either too high or too low.

6.3 Reranking Threshold

The empirical results largely confirm the initial hypothesis regarding cross-encoder similarity thresholding: chunkers characterized by a low average token count per chunk profit most from a dynamic cutoff score. Smaller, fine-grained chunks offer greater retrieval flexibility, as the system can retrieve varying numbers of highly targeted segments depending on query complexity. When paired with an appropriate similarity threshold, the retriever effectively filters out weakly scoring distractors, passing only high-confidence

contexts to the Small Language Model (SLM) without forcing low-relevance chunks to fill an arbitrary context budget.

Across the evaluated strategies, a threshold benefit boundary emerges between 100 and 200 tokens per chunk. Chunking methods producing average sizes below this threshold generally benefited from filtering at a 0.05 similarity cutoff. However, slight nuances were observed near this boundary. Dynamic Token chunking (averaging 217.5 tokens per chunk) achieved slightly higher answer correctness with a low threshold of 0.05, whereas Character chunking on linearized text (averaging 121.6 tokens per chunk) performed marginally better without thresholding (0.00).

The primary exception to this pattern was the LLM chunker, which did not align with the general trend. Characterized by unusually large average chunk sizes that typically resulted in retrieving only a single chunk per query, its sensitivity to threshold adjustments was unpredicted. Rather than signaling a genuine structural interaction, this behavior likely highlights the inherent variance and evaluation noise associated with conducting preliminary hyperparameter tuning on the smaller 15-item subset.

6.4 Query Expansion

The results partly confirm the results from Jagerman et al. (2023). Chain of Thought performs best for more configurations than HyDE. However, also the approach without query preprocessing achieves good results, hinting to the fact that modern embedding models have improved coping with the query/document asymmetry.

7 Limitations and Future Research

As this work focuses on preprocessing, chunking and query expansion, the optimization of other pipeline components is neglected. Especially the online part of the pipeline has a lot more potential for optimization. Even the maximum context size and the factor between top_k and top_n are optimizable in context of the rest of the pipeline. This could be done in an independent experiment or as part of a bigger grid search.
For the embedder, reranker, and generator, other models can be compared. For this work, only a few models were tested superficially, leaving this question open for future work. Especially for the generator, German-only models can be tested. Also larger models with a low quantization are an option. The retriever can even be combined with a sparse BM25 retriever. In previous work, this has outperformed a pure dense retriever. However, especially (but not exclusively) when switching to hybrid retrieval, the results for chunking and preprocessing might have to be re-evaluated.
For improved retrieval results, the embedding model can be fine-tuned. As the topic is very specific and the number of documents is very limited, this could boost retrieval quality by a great margin. Furthermore, the query preprocessing can be more precise with fine-tuned embeddings. Query2doc achieves a performance gain over HyDE by fine-tuning an embedding model on extended queries.
Furthermore, the components optimized in the first four experiments (preprocessor, chunking parameters, context threshold and query expansion) can be further improved in a grid search. As the experiments were conducted sequentially due to evaluation resource constraints, there is no guarantee that this thesis found the best pipeline configuration. The best chunk size for RAG is for example heavily dependent on retrieval threshold,

which was only investigated after the chunker experiment here. With more computational and monetary resources, future research could improve the outcome by finding a better combination of components and settings.
Future research could investigate all of the suggested improvement For chatbot deployment, the repository can be made more performant using lazy imports, i. e., moving imports into methods. Thus, imports are only triggered when really needed. By using vllm with a locally saved model instead of HuggingFace, there is an even greater potential gain in performance.

7.1 Main Experiment

- Whole Table and Paragraph for Markdown with similar results because similar strategies: paragraph splitting for markdown often splits at table/module borders and wholeparagraph splits at paragraph level for text documents[see example]
- both good at context recall because just has to retrieve the one or two right tables/paragraphs
- better than character chunking and SplitTable because keep tables together, LLMs often partly trained on markdown data and can handle as trained, worse if split inside the tables, also harder for the retriever to find right chunks as relevant table is split into several chunks hard to find in structured table format, resulting in low context recall
-

7.2 Evaluation Analysis

8 Conclusion

Declaration of AI usage

I, Daniel Gelderblom, Matrikelnummer 3131057, hereby confirm that I have read and acknowledged the AI policy of the Institute for Linguistics, HHU (June 2025).

I hereby declare that for my thesis with the title Optimizing RAG Pipelines for Small Language Models, supervised by Prof. Dr. Wiebke Peterse and David Arps, I have used AI. The following list contains all tools that I have used as well as a short description of how and for what purpose these tools were used.

Name of the tools	version/date	Purpose
Google Gemini	gemini 3.1 pro and 3.5 flash	Finding literature (always checked and read thoroughly before use), sharpening ideas for the pipeline, suggesting qa-pairs for the evaluation dataset
Google Antigravity with Claude Sonnet	Antigravity IDE + Claude Sonnet 4.6	Implementation and documentation of pipeline and experiments, prompted with intended architecture and exact content

References

Akram, Mohammad Kalim, Saba Sturua, Nastia Havriushenko, Quentin Herreros, Michael Günther, Maximilian Werk, and Han Xiao (2026). "jina-embeddings-v5-text: Task-Targeted Embedding Distillation". In: *arXiv preprint arXiv:2602.15547*.

Anthropic (Sept. 2024). *Contextual Retrieval*. `https://www.anthropic.com/news/contextual-retrieval`. Anthropic News & Announcements. Accessed: 2026-05-08.

Bennani, Sofia and Charles Moslonka (2026). "A Systematic Analysis of Chunking Strategies for Reliable Question Answering". In: *European Conference on Information Retrieval*. Springer, pp. 68–73.

Chen, Si-An, Lesly Miculicich, Julian M Eisenschlos, Zifeng Wang, Zilong Wang, Yanfei Chen, Yasuhisa Fujii, Hsuan-Tien Lin, Chen-Yu Lee, and Tomas Pfister (2024). "Tablerag: Million-token table understanding with language models". In: *Advances in Neural Information Processing Systems* 37, pp. 74899–74921.

Chen, Wei-Lin, Zhepei Wei, Xinyu Zhu, Shi Feng, and Yu Meng (2025). "Do LLM evaluators prefer themselves for a reason?" In: *arXiv preprint arXiv:2504.03846*.

Dai, Zihang, Zhilin Yang, Yiming Yang, Jaime G Carbonell, Quoc Le, and Ruslan Salakhutdinov (2019). "Transformer-xl: Attentive language models beyond a fixed-length context". In: *Proceedings of the 57th annual meeting of the association for computational linguistics*, pp. 2978–2988.

Deep Search Team (Aug. 2024). *Docling Technical Report*. Tech. rep. Version 1.0.0. DOI: `10.48550/arXiv.2408.09869`. eprint: `2408.09869`. URL: `https://arxiv.org/abs/2408.09869`.

Devlin, Jacob, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova (2019). "Bert: Pre-training of deep bidirectional transformers for language understanding". In: *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pp. 4171–4186.

Douze, Matthijs, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou (2024). "The Faiss library". In: arXiv: `2401.08281 [cs.LG]`.

Duarte, André V, João DS Marques, Miguel Graça, Miguel Freire, Lei Li, and Arlindo L Oliveira (2024). "Lumberchunker: Long-form narrative document segmentation". In: *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 6473–6486.

Es, Shahul, Jithin James, Luis Espinosa Anke, and Steven Schockaert (2024). "Ragas: Automated evaluation of retrieval augmented generation". In: *Proceedings of the 18th conference of the european chapter of the association for computational linguistics: system demonstrations*, pp. 150–158.

Fan, Tianyu, Jingyuan Wang, Xubin Ren, and Chao Huang (2025). "Minirag: Towards extremely simple retrieval-augmented generation". In: *arXiv preprint arXiv:2501.06713*.

Gao, Luyu, Xueguang Ma, Jimmy Lin, and Jamie Callan (2023). "Precise zero-shot dense retrieval without relevance labels". In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1762–1777.

Günther, Michael, Isabelle Mohr, Daniel James Williams, Bo Wang, and Han Xiao (2024). "Late chunking: contextual chunk embeddings using long-context embedding models". In: *arXiv preprint arXiv:2409.04701*.

Guu, Kelvin, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang (2020). "Retrieval augmented language model pre-training". In: *International conference on machine learning*. PMLR, pp. 3929–3938.

Jagerman, Rolf, Honglei Zhuang, Zhen Qin, Xuanhui Wang, and Michael Bendersky (2023). "Query expansion by prompting large language models". In: *arXiv preprint arXiv:2305.03653*.

Kamradt, Greg (2024). *5 Levels of Text Splitting: A tutorial on different chunking strategies for RAG and LLM applications*. `https://github.com/FullStackRetrieval-com/RetrievalTutorials`. Accessed: 2026-05-25.

Karpukhin, Vladimir, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih (2020). "Dense passage retrieval for open-domain question answering". In: *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)*, pp. 6769–6781.

Leng, Quinn, Jacob Portes, Sam Havens, Matei Zaharia, and Michael Carbin (2024). "Long context rag performance of large language models". In: *arXiv preprint arXiv:2411.03538*.

Lewis, Patrick, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. (2020). "Retrieval-augmented generation for knowledge-intensive nlp tasks". In: *Advances in neural information processing systems* 33, pp. 9459–9474.

Liu, Nelson F, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang (2024). "Lost in the middle: How language models use long contexts". In: *Transactions of the association for computational linguistics* 12, pp. 157–173.

Liu, Yang, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu (2023). "G-eval: NLG evaluation using gpt-4 with better human alignment". In: *Proceedings of the 2023 conference on empirical methods in natural language processing*, pp. 2511–2522.

Min, Dehai, Nan Hu, Rihui Jin, Nuo Lin, Jiaoyan Chen, Yongrui Chen, Yu Li, Guilin Qi, Yun Li, Nijun Li, et al. (2024). "Exploring the impact of table-to-text methods on augmenting llm-based question answering with domain hybrid data". In: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 6: Industry Track)*, pp. 464–482.

Nogueira, Rodrigo and Kyunghyun Cho (2019). "Passage Re-ranking with BERT". In: *arXiv preprint arXiv:1901.04085*.

OpenAI (2020). *GPT-3 Large Language Model*. Computer software. URL: https://openai.com.

Qu, Renyi, Ruixuan Tu, and Forrest Bao (2025). "Is semantic chunking worth the computational cost?" In: *Findings of the Association for Computational Linguistics: NAACL 2025*, pp. 2155–2177.

Qwen (2026). *Qwen3.5-2B*. https://huggingface.co/Qwen/Qwen3.5-2B. Model repository.

Si, Jacob, Mike Qu, Michelle Lee, and Yingzhen Li (2025). "TabRAG: Tabular Document Retrieval via Structured Language Representations". In: *arXiv preprint arXiv:2511.06582*.

Wang, Liang, Nan Yang, and Furu Wei (2023). "Query2doc: Query expansion with large language models". In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 9414–9423.

Yu, Xiaohan, Pu Jian, and Chong Chen (2025). "Tablerag: A retrieval augmented generation framework for heterogeneous document reasoning". In: *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 14074–14093.

Zheng, Lianmin, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. (2023). "Judging llm-as-a-judge with mt-bench and chatbot arena". In: *Advances in neural information processing systems* 36, pp. 46595–46623.

Zhou, Yongjie, Shuai Wang, Bevan Koopman, and Guido Zuccon (2026). "Beyond Chunk-Then-Embed: A Comprehensive Taxonomy and Evaluation of Document Chunking Strategies for Information Retrieval". In: *arXiv preprint arXiv:2602.16974*.

.1 Query Expansion

HyDE:

```
Du bist ein hilfreicher Assistent.
Schreibe einen kurzen, sachlichen Textabschnitt (2-4 Sätze),
der die folgende Frage beantwortet.
Antworte ausschließlich mit dem Textabschnitt - ohne Einleitung,
Überschrift oder sonstige Erklärungen.
```

Query2Doc:

```
Beantworte die folgende Anfrage:
{query}
Gib die Begründung an, bevor du antwortest.
```